

XS WALLET

Guia de Implementação para Desenvolvedores

Plano de 10 Semanas - 6 Fases

Documento	Guia de Implementação
Versão	1.0.0
Baseado em	Especificação Técnica v0.2.0
Data	Janeiro 2026
Público-Alvo	Desenvolvedores, Implementadores
Duração Estimada	10 semanas

1. Visão Geral do Projeto

1.1 Objetivo

Implementar uma carteira desktop self-custody com atomic swaps entre BTC, Liquid e Lightning Network, seguindo os princípios de **Zero Trust** e **Crash Recovery**.

1.2 Arquitetura Alvo

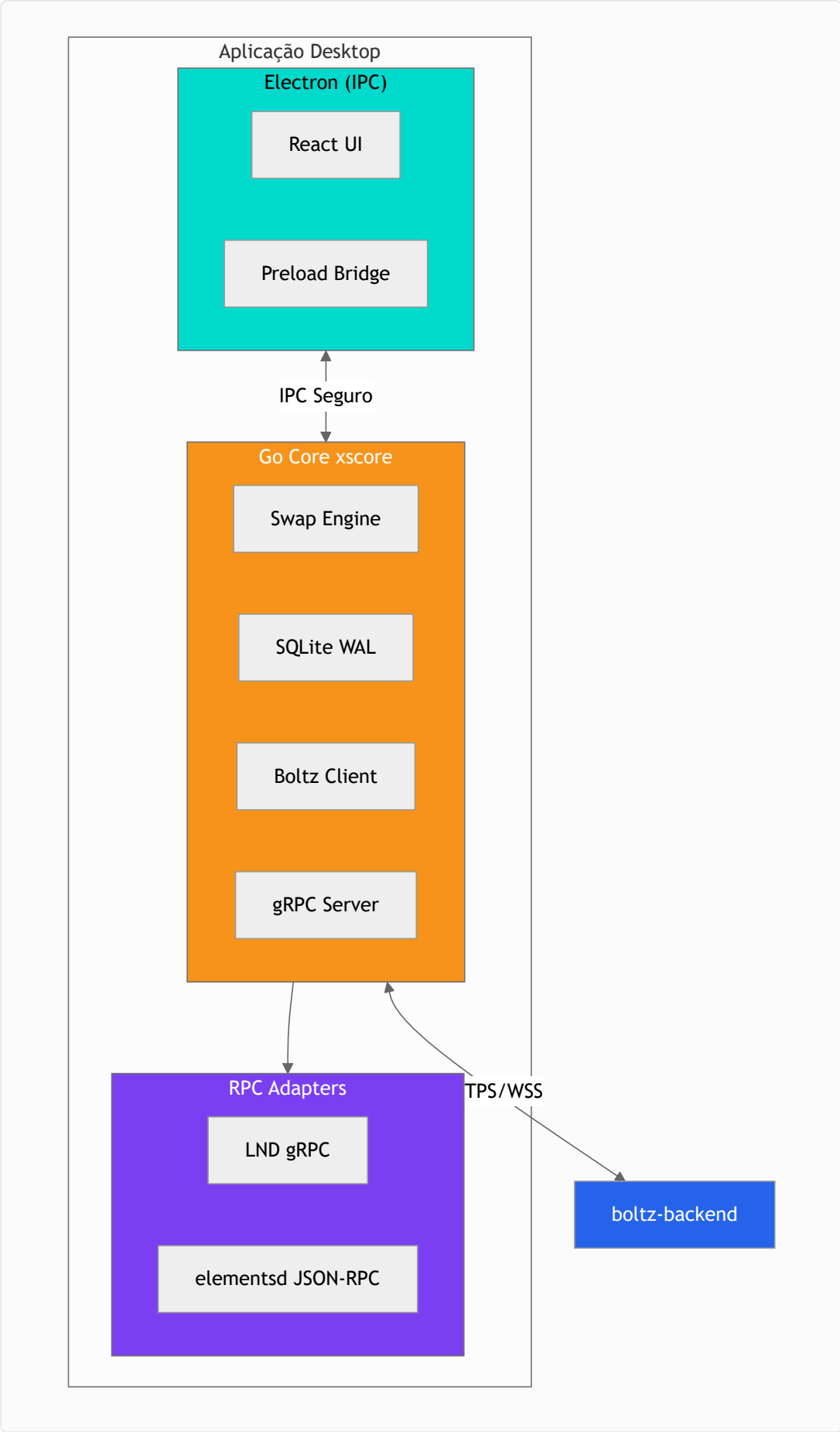


Figura 1.1: Arquitetura Alvo

1.3 Estado Atual vs Alvo

Componente	Estado Atual	Alvo
------------	--------------	------

Frontend	React + Vite (HTTP via api-bridge)	Electron + IPC
Go Core	✓ Implementado (parcial)	Completo com todos orchestrators
Swaps	Submarine (parcial)	Submarine + Reverse + Chain
MuSig2	Não implementado	Claim/refund cooperativo
Adapters	Apenas bitcoind RPC	LND gRPC + elementsd JSON-RPC

Tabela 1.1: Gap Analysis

2. Roadmap de Implementação

2.1 Visão Geral das Fases

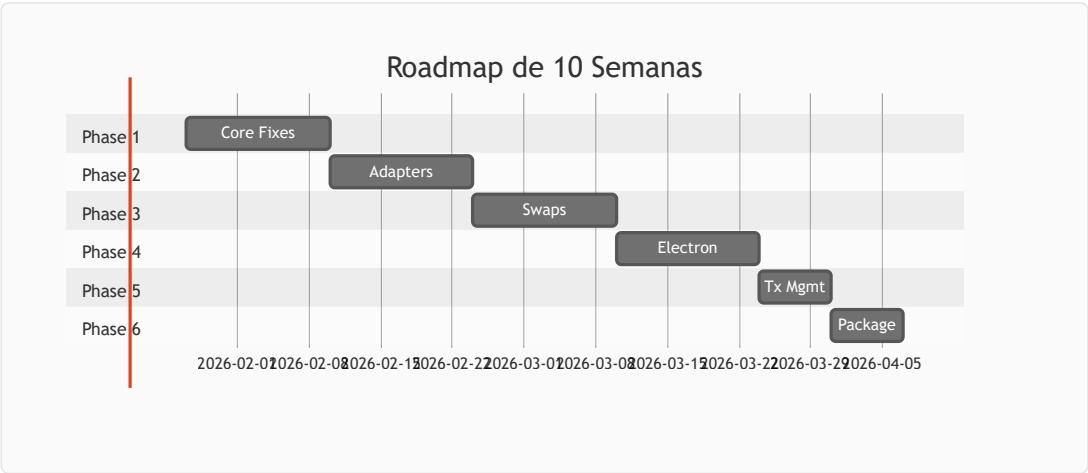


Figura 2.1: Timeline do Projeto

2.2 Dependency Graph

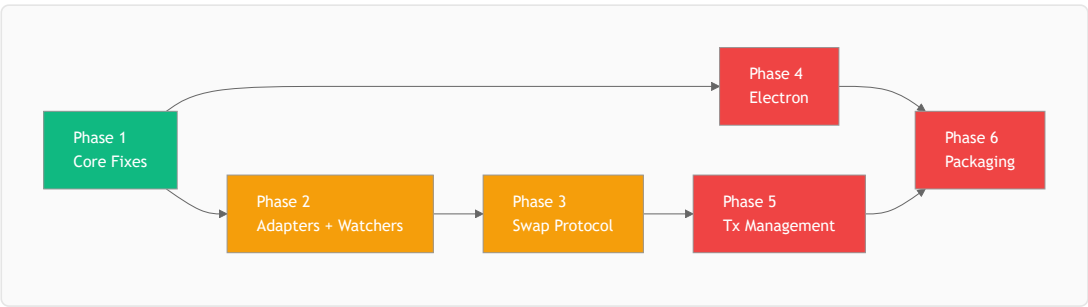


Figura 2.2: Dependências entre Fases

2.3 Milestones e Artefatos de Demo

Phase	Semana	Milestone	Artefato de Demo
1	2	Core Stable	DB schema migrado + vault unlock funcional com lockout
2	4	Adapters Ready	Watchers recebendo eventos reais (regtest) com retry logs
3	6	Swap E2E	1 swap Submarine completo com crash-recovery demonstrável
4	8	Desktop Ready	App Electron funcionando via IPC (sem api-bridge)
5	9	Tx Complete	RBF bump em swap travado + fee estimation UI
6	10	Release Candidate	Installer assinado + E2E tests passando
Buffer	11	Hardening Week	Bug fixes, polish, security review final

Tabela 2.1: Milestones com Artefatos Demonstráveis

2.4 Gestão de Riscos e Buffers

⚠ Nota: 10 semanas para uma wallet crypto self-custody é um timeline agressivo. Incluímos 10-15% de buffer (Semana 11) para mitigar riscos.

Risco	Probabilidade	Impacto	Mitigação
MuSig2 implementação complexa	Média	Alto	Fallback para script-path claim; usar <code>btcd/btcec</code>
Boltz API downtime/changes	Baixa	Alto	Self-hosted boltz-backend em regtest; mocks para CI
Electron security issues	Média	Crítico	Security Gate obrigatório antes de Phase 4
Integração LND/Elements	Média	Médio	Polar/Nigiri para ambiente controlado

Tabela 2.2: Matriz de Riscos

3. Phase 1: Core Fixes (Semanas 1-2)

✓ **Objetivo:** Consolidar a base de dados e corrigir bugs críticos antes de adicionar funcionalidades.

3.1 QuoteSwap Kind/Chain Mapping

Problema

Arquivo `core/internal/server/swap_service.go` (linhas 42-45) hardcodea `BTC→LN` e usa `req.Kind.String()` que não casa com `provider.SwapKind`.

Solução

```
// Criar helpers de conversão
func protoKindToProvider(k pb.SwapKind) provider.SwapKind {
    switch k {
        case pb.SwapKind_SWAP_KIND_SUBMARINE:
            return provider.SwapKindSubmarine
        case pb.SwapKind_SWAP_KIND_REVERSE:
            return provider.SwapKindReverse
        case pb.SwapKind_SWAP_KIND_CHAIN:
            return provider.SwapKindChain
        default:
            return ""
    }
}

func protoChainToProvider(from, to pb.Chain) (provider.Chain, provider.Chain) {
    // Mapear pb.Chain_CHAIN_BTC → provider.ChainBTC, etc
}
```

Arquivos Afetados

- `core/internal/server/swap_service.go` - adicionar helpers
- `core/internal/server/swap_service.go` - atualizar `QuoteSwap()`

3.2 Schema Unificado

Problema

Existem dois schemas divergentes:

- `core/internal/db/db.go` (~35 colunas)
- `database/schema.sqlite.sql` (40+ colunas)

Solução

Consolidar em `core/internal/db/db.go` adicionando colunas faltantes:

```
-- Adicionar ao schema em db.go
provider TEXT NOT NULL DEFAULT 'boltz',
provider_url TEXT NOT NULL,
from_asset TEXT NOT NULL,
to_asset TEXT NOT NULL,
boltz_status TEXT,
boltz_raw TEXT,
swap_tree_hex TEXT,
tap_tweak32_hex TEXT,
encrypted_preimage BLOB, -- NOVO: substituir preimage_hex
lockup_seen_mempool_at TEXT,
lockup_confirmed_height INTEGER,
claim_chain TEXT,
claim_seen_mempool_at TEXT,
claim_confirmed_height INTEGER,
refund_chain TEXT,
refund_seen_mempool_at TEXT,
refund_confirmed_height INTEGER
```

Migração

```
-- migrations/001_add_spec_columns.sql
ALTER TABLE swaps ADD COLUMN encrypted_preimage BLOB;
ALTER TABLE swaps ADD COLUMN from_asset TEXT;
ALTER TABLE swaps ADD COLUMN to_asset TEXT;
-- ... etc

-- Criar tabela vault_lockout
CREATE TABLE vault_lockout (
  id INTEGER PRIMARY KEY CHECK(id = 1),
  attempts INTEGER DEFAULT 0,
  last_attempt_at TEXT,
  locked_until TEXT
);
```

Arquivos Afetados

- `core/internal/db/db.go` - atualizar schema
- `database/schema.sqlite.sql` - REMOVER
- `migrations/001_add_spec_columns.sql` - CRIAR

3.3 Preimage Criptografada

⚠ CRÍTICO: A preimage R atualmente é salva em plaintext no DB. Isso viola o spec de segurança.

Implementação

Arquivo: `core/internal/vault/vault.go`

```
// Adicionar métodos ao Vault
func (v *Vault) EncryptPreimage(preimage []byte) ([]byte, error) {
  v.mu.RLock()
  defer v.mu.RUnlock()

  if !v.unlocked || v.seed == nil {
    return nil, ErrVaultLocked
  }

  // Derivar chave de criptografia do seed
  key := derivePreimageKey(v.seed)

  // Criptografar com AES-256-GCM
  nonce, ciphertext, err := encrypt(preimage, key[:])
  if err != nil {
    return nil, err
  }

  // Retornar nonce || ciphertext
  return append(nonce, ciphertext...), nil
}

func (v *Vault) DecryptPreimage(encrypted []byte) ([]byte, error) {
  v.mu.RLock()
  defer v.mu.RUnlock()

  if !v.unlocked || v.seed == nil {
    return nil, ErrVaultLocked
  }

  // Separar nonce e ciphertext
  nonce := encrypted[:nonceLength]
  ciphertext := encrypted[nonceLength:]

  key := derivePreimageKey(v.seed)
  return decrypt(ciphertext, nonce, key[:])
}
```

Arquivo: `core/internal/swap/engine.go`

```
// Alterar Create() para usar encrypted_preimage
func (e *Engine) Create(...) (*Swap, error) {
  // ... existing code ...
```



```

// Criptografar preimage
encryptedPreimage, err := vault.EncryptPreimage(preimage)
if err != nil {
    return nil, err
}

// Salvar encrypted_preimage em vez de preimage_hex
_, err = e.db.ExecContext(ctx, `
    INSERT INTO swaps (id, ..., encrypted_preimage, preimage_hash_hex, ...)
    VALUES (?, ..., ?, ?, ...)
`, id, ..., encryptedPreimage, hex.EncodeToString(hash), ...)
}

```

Arquivos Afetados

- `core/internal/vault/vault.go` - adicionar `EncryptPreimage/DecryptPreimage`
- `core/internal/swap/engine.go` - usar `encrypted_preimage`
- `core/internal/db/db.go` - schema com `encrypted_preimage`

3.4 PIN Lockout Persistente

Implementação

Arquivo: `core/internal/vault/lockout.go`

```

package vault

import (
    "database/sql"
    "time"
    "github.com/xs-wallet/xscore/internal/db"
)

type LockoutManager struct {
    db          *db.DB
    maxAttempts int        // 10
    lockoutTime time.Duration // 30 min
}

func NewLockoutManager(database *db.DB) *LockoutManager {
    return &LockoutManager{
        db:          database,
        maxAttempts: 10,
        lockoutTime: 30 * time.Minute,
    }
}

func (l *LockoutManager) RecordAttempt(success bool) error {
    if success {
        // Reset attempts
        _, err := l.db.Exec(`
            UPDATE vault_lockout SET attempts = 0, locked_until = NULL
            WHERE id = 1
        `)
        return err
    }

    // Increment attempts
    _, err := l.db.Exec(`
        INSERT INTO vault_lockout (id, attempts, last_attempt_at)
        VALUES (1, 1, datetime('now'))
        ON CONFLICT(id) DO UPDATE SET
            attempts = attempts + 1,
            last_attempt_at = datetime('now'),
            locked_until = CASE
                WHEN attempts + 1 >= ? THEN datetime('now', '+30 minutes')
                ELSE locked_until
            END
    `, l.maxAttempts)
    return err
}

func (l *LockoutManager) CanAttempt() (bool, time.Duration, error) {
    var attempts int
    var lockedUntil sql.NullString

```

```

err := l.db.QueryRow(`
    SELECT attempts, locked_until FROM vault_lockout WHERE id = 1
`).Scan(&attempts, &lockedUntil)

if err == sql.ErrNoRows {
    return true, 0, nil
}
if err != nil {
    return false, 0, err
}

if lockedUntil.Valid {
    until, _ := time.Parse("2006-01-02 15:04:05", lockedUntil.String)
    if time.Now().Before(until) {
        return false, time.Until(until), nil
    }
}

return true, 0, nil
}

```

Integração no Vault:

```

// Em vault.go
func (v *Vault) Unlock(pin string) (string, error) {
    // Verificar lockout
    canAttempt, waitTime, err := v.lockout.CanAttempt()
    if err != nil {
        return "", err
    }
    if !canAttempt {
        return "", fmt.Errorf("locked out for %v", waitTime)
    }

    // ... existing unlock logic ...

    // Registrar resultado
    v.lockout.RecordAttempt(err == nil)

    return sessionID, err
}

```

Arquivos Afetados

- `core/internal/vault/lockout.go` - CRIAR
- `core/internal/vault/vault.go` - integrar LockoutManager
- `core/internal/db/db.go` - adicionar tabela vault_lockout

3.5 Checklist Phase 1

Tarefa	Arquivos	Status
QuoteSwap fix	swap_service.go	Pendente
Schema unificado	db.go, migrations/	Pendente
Preimage encrypted	vault.go, engine.go	Pendente
PIN lockout	lockout.go, vault.go	Pendente

3.6 Definition of Done - Phase 1

✓ Phase 1 está DONE quando:

- ☒ Schema SQLite unificado criado via migrations (não mais divergência)
- ☒ QuoteSwap() aceita todos os pares: BTC↔LN, LN↔BTC, BTC↔Liquid
- ☒ Preimages criptografadas no DB (verificável via hex dump)
- ☒ PIN lockout funcional: 10 tentativas → 30min lock (testável)
- ☒ Todos os unit tests passando: `go test ./...`

4. Phase 2: Adapters + Watchers (Semanas 3-4)

✓ **Objetivo:** Base funcional (adapters + watchers) para orchestrators consumirem dados reais.

4.1 LND Adapter Mínimo

Arquivo: `core/internal/adapters/lnd/client.go`

```
package lnd

import (
    "context"
    "google.golang.org/grpc"
    "github.com/lightningnetwork/lnd/lnrpc"
)

type Client struct {
    conn      *grpc.ClientConn
    lightning lnrpc.LightningClient
}

func NewClient(address string, tlsCert, macaroon []byte) (*Client, error) {
    // Setup TLS + macaroon credentials
    creds := credentials.NewTLS(&tls.Config{
        Certificates: []tls.Certificate{cert},
    })

    conn, err := grpc.Dial(address, grpc.WithTransportCredentials(creds))
    if err != nil {
        return nil, err
    }

    return &Client{
        conn:      conn,
        lightning: lnrpc.NewLightningClient(conn),
    }, nil
}

// Mínimo viável
func (c *Client) GetInfo(ctx context.Context) (*lnrpc.GetInfoResponse, error) {
    return c.lightning.GetInfo(ctx, &lnrpc.GetInfoRequest{})
}

func (c *Client) PayInvoice(ctx context.Context, bolt11 string) (*lnrpc.SendResponse, error) {
    return c.lightning.SendPaymentSync(ctx, &lnrpc.SendRequest{
        PaymentRequest: bolt11,
    })
}

func (c *Client) AddInvoice(ctx context.Context, amount int64, memo string) (*lnrpc.AddInvoiceResponse, error) {
    return c.lightning.AddInvoice(ctx, &lnrpc.Invoice{
        Value: amount,
        Memo:  memo,
    })
}

// Stream de invoices
func (c *Client) SubscribeInvoices(ctx context.Context) (<chan *lnrpc.Invoice, error) {
    stream, err := c.lightning.SubscribeInvoices(ctx, &lnrpc.InvoiceSubscription{})
    if err != nil {
        return nil, err
    }

    invoices := make(chan *lnrpc.Invoice)
    go func() {
        defer close(invoices)
        for {
            inv, err := stream.Recv()
            if err != nil {
                return
            }
            invoices <- inv
        }
    }()
}
```

```

    return invoices, nil
}

func (c *Client) LookupInvoice(ctx context.Context, paymentHash string) (*lnrpc.Invoice, error) {
    hash, _ := hex.DecodeString(paymentHash)
    return c.lightning.LookupInvoice(ctx, &lnrpc.PaymentHash{
        RHash: hash,
    })
}

```

4.2 BTC/Liquid Adapter Mínimo

Arquivo: core/internal/adapters/bitcoin/client.go

```

package bitcoin

import (
    "bytes"
    "encoding/json"
    "net/http"
)

type Client struct {
    rpcURL    string
    user      string
    password  string
    client    *http.Client
}

func NewClient(rpcURL, user, password string) *Client {
    return &Client{
        rpcURL:    rpcURL,
        user:      user,
        password:  password,
        client:    &http.Client{Timeout: 30 * time.Second},
    }
}

type RPCRequest struct {
    JSONRPC string    `json:"jsonrpc"`
    Method  string    `json:"method"`
    Params  []interface{} `json:"params"`
    ID      int       `json:"id"`
}

func (c *Client) call(method string, params ...interface{}) (json.RawMessage, error) {
    req := RPCRequest{
        JSONRPC: "1.0",
        Method:  method,
        Params:  params,
        ID:      1,
    }

    body, _ := json.Marshal(req)
    httpReq, _ := http.NewRequest("POST", c.rpcURL, bytes.NewReader(body))
    httpReq.SetBasicAuth(c.user, c.password)
    httpReq.Header.Set("Content-Type", "application/json")

    resp, err := c.client.Do(httpReq)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()

    var rpcResp struct {
        Result json.RawMessage `json:"result"`
        Error  *struct {
            Message string `json:"message"`
        } `json:"error"`
    }

    json.NewDecoder(resp.Body).Decode(&rpcResp)
    if rpcResp.Error != nil {
        return nil, fmt.Errorf("RPC error: %s", rpcResp.Error.Message)
    }

    return rpcResp.Result, nil
}

```

```

}

// Mínimo viável
func (c *Client) GetBlockHeight(ctx context.Context) (int64, error) {
    result, err := c.call("getblockcount")
    if err != nil {
        return 0, err
    }

    var height int64
    json.Unmarshal(result, &height)
    return height, nil
}

func (c *Client) GetTransaction(ctx context.Context, txid string) (*Transaction, error) {
    result, err := c.call("getrawtransaction", txid, true)
    if err != nil {
        return nil, err
    }

    var tx Transaction
    json.Unmarshal(result, &tx)
    return &tx, nil
}

func (c *Client) BroadcastTransaction(ctx context.Context, hex string) (string, error) {
    result, err := c.call("sendrawtransaction", hex)
    if err != nil {
        return "", err
    }

    var txid string
    json.Unmarshal(result, &txid)
    return txid, nil
}

func (c *Client) GetConfirmations(ctx context.Context, txid string) (int, error) {
    tx, err := c.GetTransaction(ctx, txid)
    if err != nil {
        return 0, err
    }
    return tx.Confirmations, nil
}

```

Arquivo: core/internal/adapters/liquid/client.go (mesmo interface)

4.3 Chain Watcher

Arquivo: core/internal/watcher/chain_watcher.go

```

package watcher

import (
    "context"
    "time"
    "github.com/xs-wallet/xscore/internal/adapters/bitcoin"
    "github.com/xs-wallet/xscore/internal/adapters/liquid"
    "github.com/xs-wallet/xscore/internal/swap"
)

type ChainWatcher struct {
    btc    *bitcoin.Client
    liquid *liquid.Client
    engine *swap.Engine
}

func NewChainWatcher(btc *bitcoin.Client, liquid *liquid.Client, engine *swap.Engine) *ChainWatcher {
    return &ChainWatcher{
        btc:    btc,
        liquid: liquid,
        engine: engine,
    }
}

func (w *ChainWatcher) WatchTransaction(ctx context.Context, chain, txid string) error {
    ticker := time.NewTicker(30 * time.Second)
    defer ticker.Stop()

    for {

```

```

select {
case ←ctx.Done():
    return ctx.Err()
case ←ticker.C:
    var confirmations int
    var err error

    if chain == "BTC" {
        confirmations, err = w.btc.GetConfirmations(ctx, txid)
    } else {
        confirmations, err = w.liquid.GetConfirmations(ctx, txid)
    }

    if err ≠ nil {
        continue
    }

    if confirmations ≥ 1 {
        // Trigger reconcile
        go w.reconcileByTxid(ctx, txid)
        return nil
    }
}
}

func (w *ChainWatcher) GetBlockHeight(ctx context.Context, chain string) (int64, error) {
    if chain == "BTC" {
        return w.btc.GetBlockHeight(ctx)
    }
    return w.liquid.GetBlockHeight(ctx)
}

```

4.4 LN Watcher

Arquivo: core/internal/watcher/ln_watcher.go

```

package watcher

import (
    "context"
    "github.com/xs-wallet/xscore/internal/adapters/lnd"
    "github.com/xs-wallet/xscore/internal/swap"
)

type LNWatcher struct {
    lnd    *lnd.Client
    engine *swap.Engine
}

func NewLNWatcher(lnd *lnd.Client, engine *swap.Engine) *LNWatcher {
    return &LNWatcher{
        lnd:    lnd,
        engine: engine,
    }
}

func (w *LNWatcher) SubscribeInvoices(ctx context.Context) error {
    invoices, err := w.lnd.SubscribeInvoices(ctx)
    if err ≠ nil {
        return err
    }

    for {
        select {
        case ←ctx.Done():
            return ctx.Err()
        case inv := ←invoices:
            if inv.State == lnrpc.Invoice_SETTLED {
                // Trigger reconcile
                go w.reconcileByPaymentHash(ctx, hex.EncodeToString(inv.RHash))
            }
        }
    }
}

```

4.5 Watcher + WS Integration

Arquivo: core/internal/watcher/watcher.go

```
package watcher

import (
    "context"
    "github.com/xs-wallet/xscore/internal/boltz"
    "github.com/xs-wallet/xscore/internal/swap"
)

type Watcher struct {
    engine      *swap.Engine
    chainWatch  *ChainWatcher
    lnWatch     *LNWatcher
}

func NewWatcher(engine *swap.Engine, chain *ChainWatcher, ln *LNWatcher) *Watcher {
    return &Watcher{
        engine:      engine,
        chainWatch:  chain,
        lnWatch:     ln,
    }
}

// Handler para WS updates do Boltz
func (w *Watcher) HandleWSUpdate(swapID, boltzStatus string) {
    // 1. Buscar swap
    swap, err := w.engine.Get(context.Background(), swapID)
    if err != nil {
        log.Printf("Failed to get swap %s: %v", swapID, err)
        return
    }

    // 2. Normalizar status usando boltz.Normalize existente
    mapping := boltz.Normalize(boltzStatus, swap.Kind)

    // 3. Se requer ação, reconciliar
    if mapping.Action != boltz.ActionNone {
        go w.reconcileSwap(context.Background(), swap)
    }
}

func (w *Watcher) reconcileSwap(ctx context.Context, swap *swap.Swap) {
    // Lógica de reconciliação baseada em estado
    // Será expandida na Phase 3
}
```

4.6 Node Manager Mínimo

O Node Manager é pré-requisito para Phase 4 (Electron) e Phase 6 (Packaging). Implementação mínima necessária nesta fase:

Arquivo: core/internal/nodemanager/manager.go

```
package nodemanager

// Start inicia um node após verificar binário
func (nm *NodeManager) Start(ctx context.Context, name string) error {
    // 1. Verificar hash do binário contra manifest
    if err := nm.verifyBinary(name); err != nil {
        nm.nodes[name].Status = StatusError
        return fmt.Errorf("binary verification failed: %w", err)
    }

    // 2. Spawn processo
    // 3. Aguardar health check
    return nil
}

// Stop encerra gracefully via RPC, então SIGTERM
func (nm *NodeManager) Stop(name string) error { ... }

// HealthCheck verifica status via RPC (getinfo/getblockchaininfo)
func (nm *NodeManager) HealthCheck(name string) (bool, error) { ... }
```

Funcionalidades Mínimas

- **start()** - Spawn com verificação de binário
- **stop()** - Graceful shutdown via RPC → SIGTERM
- **healthCheck()** - Polling via gRPC/JSON-RPC
- **verifyBinary()** - SHA256 hash check contra manifest assinado

4.7 Checklist Phase 2

Tarefa	Arquivos	Status
LND Adapter	lnd/client.go	Pendente
BTC Adapter	bitcoin/client.go	Pendente
Liquid Adapter	liquid/client.go	Pendente
Chain Watcher	chain_watcher.go	Pendente
LN Watcher	ln_watcher.go	Pendente
Watcher+WS	watcher.go	Pendente
Node Manager Mínimo	nodemanager/manager.go, verify.go	Pendente

4.8 Definition of Done - Phase 2

✓ Phase 2 está DONE quando:

- ✓ LND adapter conecta e executa `GetInfo()` em regtest
- ✓ Watchers recebendo eventos reais com retry (exponential backoff)
- ✓ WebSocket Boltz recebendo status updates e disparando reconcile
- ✓ Logs estruturados com `slog` para todas as operações de adapter
- ✓ **Node Manager mínimo:** start/stop/health + binary verify funcionando
- ✓ Integration tests passando contra Polar/Nigiri

5. Phase 3: Swap Protocol (Semanas 5-6)

✓ **Objetivo:** Implementar todos os tipos de swap (Submarine, Reverse, Chain) com MuSig2.

5.1 Submarine Auto-Lock

Problema

Atualmente `CreateFromQuote()` deixa o swap em estado `open` sem `lockup_address`. O usuário precisa chamar `Lock()` manualmente.

Solução

```
// core/internal/swap/submarine.go
func (so *SubmarineOrchestrator) CreateFromQuote(ctx, quoteID) (*Swap, error) {
    // Validar quote
    quote, err := so.quotes.GetQuote(ctx, quoteID)
    if err != nil {
        return nil, err
    }

    // Criar swap
    swap, err := so.engine.Create(ctx, KindSubmarine, "regtest", 0)
    if err != nil {
        return nil, err
    }

    // AUTO-LOCK imediato
    swap, err = so.Lock(ctx, swap.ID, quoteID)
    if err != nil {
        return nil, err
    }

    return swap, nil
}
```

5.2 Reverse Swap Orchestrator

Arquivo: `core/internal/swap/reverse.go`

```
package swap

type ReverseOrchestrator struct {
    engine *Engine
    db      *db.DB
    provider provider.Provider
    vault   *vault.Vault
}

func NewReverseOrchestrator(engine *Engine, database *db.DB, prov provider.Provider, v *vault.Vault) *ReverseOrchestrator {
    return &ReverseOrchestrator{
        engine: engine,
        db:     database,
        provider: prov,
        vault:  v,
    }
}

func (ro *ReverseOrchestrator) CreateFromQuote(ctx, quoteID) (*Swap, error) {
    // 1. Gerar preimage R
    preimage := make([]byte, 32)
    rand.Read(preimage)
    hash := sha256.Sum256(preimage)

    // 2. Criar swap
    swap, err := ro.engine.Create(ctx, KindReverse, "regtest", 0)
    if err != nil {
        return nil, err
    }
}
```

```

// 3. Criptografar e salvar preimage
encPreimage, err := ro.vault.EncryptPreimage(preimage)
if err != nil {
    return nil, err
}

// 4. Atualizar DB
_, err = ro.db.ExecContext(ctx, `
    UPDATE swaps SET encrypted_preimage = ?, preimage_hash_hex = ?
    WHERE id = ?
`, encPreimage, hex.EncodeToString(hash[:]), swap.ID)

// 5. Lock com provider
return ro.Lock(ctx, swap.ID, quoteID)
}

func (ro *ReverseOrchestrator) PayInvoice(ctx, swapID) (*Swap, error) {
    // Pagar hold invoice do provider
    // Transição: Locked → waiting
}

func (ro *ReverseOrchestrator) Claim(ctx, swapID) (*Swap, error) {
    // Tentar MuSig2 key-path claim
    // Fallback: script-path claim
}

```

5.3 Chain Swap Orchestrator

Arquivo: core/internal/swap/chain.go

```

package swap

type ChainOrchestrator struct {
    engine *Engine
    db      *db.DB
    provider provider.Provider
}

func (co *ChainOrchestrator) CreateFromQuote(ctx, quoteID) (*Swap, error) {
    // Similar ao reverse: gerar R, criar swap, lock
}

func (co *ChainOrchestrator) FundBTC(ctx, swapID) (*Swap, error) {
    // Broadcast funding tx BTC
}

func (co *ChainOrchestrator) ClaimLiquid(ctx, swapID) (*Swap, error) {
    // Claim HTLC Liquid (revela R)
}

```

5.4 P2TR Verification

Arquivo: core/internal/swap/taproot.go

```

package swap

import (
    "github.com/btcsuite/btcd/txscript"
    "github.com/btcsuite/btcd/btcec/v2"
)

// VerifyP2TRAddress rebuilds o endereço P2TR localmente e compara
func VerifyP2TRAddress(
    claimPubkey, refundPubkey []byte,
    preimageHash []byte,
    timeoutBlocks int64,
    expectedAddress string,
) error {
    // 1. Construir swap tree
    tree, err := BuildSwapTree(KindSubmarine, SwapTreeParams{
        ClaimPubkey:  claimPubkey,
        RefundPubkey: refundPubkey,
        PreimageHash: preimageHash,
        TimeoutBlocks: timeoutBlocks,
    })
}

```

```

    if err != nil {
        return err
    }

    // 2. Gerar endereço P2TR
    address, err := tree.ToAddress(&chaincfg.ReggressionNetParams)
    if err != nil {
        return err
    }

    // 3. Comparar
    if address != expectedAddress {
        return fmt.Errorf("address mismatch: got %s, want %s", address, expectedAddress)
    }

    return nil
}

type SwapTree struct {
    InternalKey *btcec.PublicKey
    Scripts     [][]byte
}

func BuildSwapTree(kind Kind, params SwapTreeParams) (*SwapTree, error) {
    // Construir scripts conforme Boltz v2 spec
    // - Claim path: OP_SIZE 32 OP_EQUAL OP_IF OP_HASH256 OP_EQUALVERIFY OP_ELSE OP_CHECKLOCKTIMEVERI
}

```

5.5 MuSig2 Implementation

Arquivo: core/internal/musig2/session.go

```

package musig2

import (
    "github.com/btcsuite/btcd/btcec/v2"
    "github.com/btcsuite/btcd/btcec/v2/schnorr/musig2"
)

type Session struct {
    ID          string
    SwapID      string
    UserKey     *btcec.PublicKey
    ProviderKey *btcec.PublicKey
    AggKey      *musig2.AggregateKey
    SecNonce    [musig2.SecNonceSize]byte // CRÍTICO: nunca expor
    PubNonce    [musig2.PubNonceSize]byte
    AggNonce    *musig2.Nonces
    PartialSig  *musig2.PartialSignature
}

func NewSession(swapID string, userPriv *btcec.PrivateKey, providerPub *btcec.PublicKey) (*Session, error) {
    // Agregar chaves (ordem lexicográfica)
    keys := []*btcec.PublicKey{userPriv.PubKey(), providerPub}
    sort.Slice(keys, func(i, j int) bool {
        return bytes.Compare(keys[i].SerializeCompressed(), keys[j].SerializeCompressed()) < 0
    })

    aggKey, err := musig2.AggregateKeys(keys, false)
    if err != nil {
        return nil, err
    }

    return &Session{
        ID:          uuid.New().String(),
        SwapID:      swapID,
        UserKey:     userPriv.PubKey(),
        ProviderKey: providerPub,
        AggKey:      aggKey,
    }, nil
}

func (s *Session) GenerateNonce() (pubNonce []byte, err error) {
    // Gerar nonce com aux randomness
    var auxRand [32]byte
    rand.Read(auxRand[:])

    secNonce, pubNonce, err := musig2.GenNonces(

```

```

        musig2.WithCustomRand(bytes.NewReader(auxRand[:])),
    )
    if err != nil {
        return nil, err
    }

    s.SecNonce = *secNonce
    copy(s.PubNonce[:], pubNonce[:])

    return s.PubNonce[:], nil
}

func (s *Session) AggregateNonces(providerNonce []byte) error {
    nonces := [][]musig2.PubNonceSize{byte{s.PubNonce}}
    var provNonce [musig2.PubNonceSize]byte
    copy(provNonce[:], providerNonce)
    nonces = append(nonces, provNonce)

    aggNonce, err := musig2.AggregateNonces(nonces)
    if err != nil {
        return err
    }

    s.AggNonce = aggNonce
    return nil
}

func (s *Session) Sign(txHash []byte, userPriv *btcec.PrivateKey) (partialSig []byte, err error) {
    var msg [32]byte
    copy(msg[:], txHash)

    partialSig, err = musig2.Sign(
        &s.SecNonce,
        userPriv,
        s.AggNonce,
        s.AggKey.FinalKey,
        msg,
    )
    if err != nil {
        return nil, err
    }

    // Cleanup secononce
    for i := range s.SecNonce {
        s.SecNonce[i] = 0
    }

    return partialSig, nil
}

func (s *Session) Cleanup() {
    // Zero sensitive data
    for i := range s.SecNonce {
        s.SecNonce[i] = 0
    }
}

```

5.6 Watcher + WebSocket Integration

Arquivo: core/internal/watcher/watcher.go

```

// Adicionar handler para WS updates
func (w *Watcher) HandleWSUpdate(swapID, boltzStatus string, tx *boltz.TxInfo) {
    // 1. Buscar swap
    swap, err := w.engine.Get(context.Background(), swapID)
    if err != nil {
        log.Printf("Failed to get swap %s: %v", swapID, err)
        return
    }

    // 2. Normalizar status usando boltz.Normalize existente
    mapping := boltz.Normalize(boltzStatus, swap.Kind)

    // 3. Se requer ação, reconciliar
    if mapping.Action != boltz.ActionNone {
        go w.reconcileSwap(context.Background(), swap)
    }
}

```

```
}  
}
```

5.7 Checklist Phase 3

Tarefa	Arquivos	Status
Auto-Lock	submarine.go	Pendente
Reverse Swap	reverse.go	Pendente
Chain Swap	chain.go	Pendente
P2TR Verification	taproot.go	Pendente
MuSig2	musig2/session.go	Pendente
Watcher+WS	watcher.go	Pendente

5.8 Definition of Done - Phase 3

✓ Phase 3 está DONE quando:

- ✓ 1 Submarine Swap E2E completo em regtest (fund → claim → complete)
- ✓ 1 Reverse Swap E2E completo em regtest (invoice → payment → claim)
- ✓ Crash-recovery demonstrável: kill -9 mid-swap → restart → swap continua
- ✓ MuSig2 key-path claim funcionando OU fallback script-path testado
- ✓ P2TR address verification passando (rebuild local = endereço Boltz)

5.9 Security Gate

🔒 GATE OBRIGATÓRIO: Antes de iniciar Phase 4 (Electron), os seguintes itens devem ser revisados:

Item	Responsável	Critério de Aprovação
Threat Model Review	Security Lead	Documento atualizado com vetores de ataque Electron
Preimage Security Audit	Dev Lead	Verificar criptografia AES-256-GCM + key derivation
IPC Whitelist Design	Security Lead	Lista de métodos permitidos no preload bridge aprovada
Dependencies Audit	DevOps	<code>npm audit</code> + <code>go mod verify</code> sem críticos

Tabela 5.1: Security Gate Checklist

6. Gerenciamento de Nodes

6.1 Gerenciamento de Ciclo de Vida

O **Node Manager** lida com verificação de binários, spawning, monitoramento de health, e shutdown graceful de nodes embarcados. Cada node tem diretórios de dados isolados por rede.

Nota: A aplicação trata **LND (gRPC)** e **elementsd (JSON-RPC)** como RPCs primários; bitcoind, quando presente, é uma dependência do runtime e não uma integração primária do app.

Node	Versão	Diretório de Dados	Health Check	Papel
LND	0.17.3	%APPDATA%/xs-wallet/nodes/lnd/{network}/	getinfo (gRPC)	RPC primário (on-chain BTC + LN via LND)
elementsd	23.2.1	%APPDATA%/xs-wallet/nodes/liquid/{network}/	getblockchaininfo (JSON-RPC)	RPC primário (Liquid/Elements)
bitcoind (opcional)	26.0	%APPDATA%/xs-wallet/nodes/btc/{network}/	getblockchaininfo (RPC)	Dependência do runtime (quando necessário pelo stack)

Tabela 6.1: Especificações de Nodes

6.2 Verificação de Binários

Binários são baixados no primeiro uso de sources oficiais (GitHub Releases, bitcoincore.org) e verificados usando checksums SHA256 de um manifest assinado.

6.2.1 Especificação de Supply Chain

Componente	Especificação
Algoritmo de Assinatura	Ed25519 (manifest.sig)
Chave Pública	Pinned no binário da aplicação (src/constants/MANIFEST_PUBKEY)
Rotação de Chave	Nova chave anunciada via release N-1; aceita ambas por 2 releases
Manifest URL	https://github.com/xs-wallet/releases/latest/nodes-manifest.json
Fallback URLs	Espelhos: IPFS (ipfs:// ...) + backup CDN
Hash Algorithm	SHA256 (hex) para cada binário
Verificação	1) Verificar sig do manifest → 2) Verificar hash do binário → 3) Extrair

Tabela 6.2: Especificação de Supply Chain

 **Cadeia de Confiança:** Sem servidor central de custódia ou execução de swaps. Apenas endpoints públicos de distribuição (GitHub Releases, IPFS). Usuário pode verificar assinatura manualmente via CLI.

6.3 Implementação do Node Manager

Arquivo: core/internal/nodemanager/manager.go

```
package nodemanager
```

```

import (
    "context"
    "os/exec"
    "time"
)

type NodeManager struct {
    nodes    map[string]*ManagedNode
    dataDir  string
    network  string
}

type ManagedNode struct {
    Name      string
    Binary    string
    Version   string
    DataDir   string
    Process   *exec.Cmd
    Health    HealthChecker
    Status    NodeStatus
}

type NodeStatus string

const (
    StatusStopped NodeStatus = "stopped"
    StatusStarting NodeStatus = "starting"
    StatusRunning  NodeStatus = "running"
    StatusError    NodeStatus = "error"
)

func NewNodeManager(dataDir, network string) *NodeManager {
    return &NodeManager{
        nodes:    make(map[string]*ManagedNode),
        dataDir:  dataDir,
        network:  network,
    }
}

func (nm *NodeManager) Start(ctx context.Context, nodeName string) error {
    node := nm.nodes[nodeName]
    if node == nil {
        return fmt.Errorf("node %s not registered", nodeName)
    }

    // 1. Verificar binário (hash check)
    if err := nm.verifyBinary(node); err != nil {
        return fmt.Errorf("binary verification failed: %w", err)
    }

    // 2. Spawn process
    node.Status = StatusStarting
    if err := nm.spawnProcess(ctx, node); err != nil {
        node.Status = StatusError
        return err
    }

    // 3. Aguardar health check
    if err := nm.waitForHealth(ctx, node, 30*time.Second); err != nil {
        nm.Stop(nodeName)
        return err
    }

    node.Status = StatusRunning
    return nil
}

func (nm *NodeManager) Stop(nodeName string) error {
    // Graceful shutdown via RPC, then SIGTERM
}

func (nm *NodeManager) HealthCheck(nodeName string) (bool, error) {
    // Chamar health checker específico do node
}

```

⚠ Rollback/Failure Mode: Se verificação de binário falhar → não executa binário; `status=error` ; loga motivo detalhado; UI mostra erro acionável com opção de retry/download manual. **Nunca executar binário não verificado.**

6.4 Checklist - Node Management

Tarefa	Arquivos	Status
Node Manager core	nodemanager/manager.go	Pendente
Binary verification	nodemanager/verify.go	Pendente
Manifest fetcher	nodemanager/manifest.go	Pendente
LND health checker	nodemanager/lnd_health.go	Pendente
Elements health checker	nodemanager/elements_health.go	Pendente

7. Phases 4-6: Resumo

7.1 Phase 4: Electron Migration (Semanas 7-8)

Entregas

- **Electron main hardening** - `electron/main.ts`
- **Preload IPC bridge** - Whitelist + validação Zod
- **Migração do frontend** - HTTP/api-bridge → IPC/gRPC

7.2 Phase 5: Tx Management (Semana 9)

Entregas

- **RBF/CPFP bumper** - `core/internal/tx/bumper.go`
- **Fee estimation**
- **Integração WalletKit/elements**
- **Triggers por estado/timeout**

7.3 Phase 6: Packaging (Semana 10)

Entregas

- **Installer/DMG/notarização** - Windows Installer (Inno Setup), macOS DMG, Notarização Apple
- **Code signing** - Authenticode + Developer ID
- **Pipeline release**
- **Testes E2E + crash recovery**

8. Guia de Desenvolvimento

8.1 Setup do Ambiente

Pré-requisitos

```
# Go 1.21+
go version

# Node.js 18+
node --version

# Docker (para regtest)
docker --version
```

Clone e Build

```
# Clone
git clone https://github.com/4LFR3Dv1/XSWallet.git
cd XSWallet

# Build Go Core
cd core
go build ./cmd/xscore
./xscore.exe --network=regtest --port=9735

# Frontend (dev)
cd ../frontend
npm install
npm run dev

# API Bridge (temporário)
cd ../api-bridge
npm install
npm start
```

8.2 Regtest Environment

```
# Subir regtest
docker compose -f test/regtest/docker-compose.yml up -d

# Verificar
docker compose -f test/regtest/docker-compose.yml ps
```

8.3 Workflow de Desenvolvimento + CI

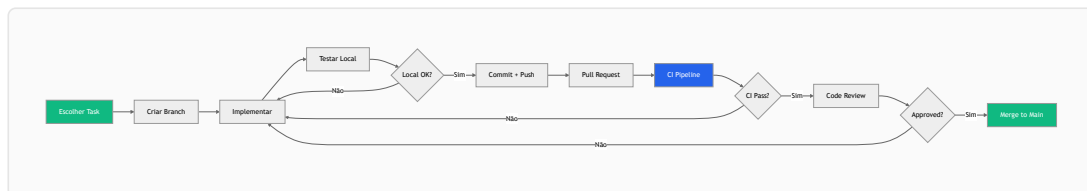


Figura 8.1: Workflow de Desenvolvimento com CI

CI Pipeline (GitHub Actions)

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps:
```

```
- uses: actions/checkout@v4
- uses: actions/setup-go@v5
  with: { go-version: '1.21' }
- run: go fmt ./... && git diff --exit-code # Lint
- run: go vet ./... # Static Analysis
- run: go test -race ./... # Unit Tests
- run: golangci-lint run # SAST

security:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: go mod verify # Deps integrity
    - uses: securego/gosec@latest # Security scan
```

Release Checklist

Step	Comando/Ação	Critério
1. Version Bump	<code>git tag v0.x.0</code>	Semantic versioning
2. CHANGELOG	Atualizar CHANGELOG.md	Breaking changes destacados
3. E2E Tests	<code>go test -tags=e2e ./...</code>	100% pass em regtest
4. Security Scan	<code>gosec ./...</code>	Zero HIGH/CRITICAL
5. Build	<code>npm run build:electron</code>	Binário assinado gerado
6. Publish	GitHub Release + Manifest	SHA256 + Ed25519 sig

Tabela 8.1: Release Checklist

8.4 Convenções de Código

Go

- Seguir `gofmt`
- Comentários em português ou inglês (consistente por arquivo)
- Testes em `*_test.go`

TypeScript/React

- ESLint + Prettier
- Componentes funcionais com hooks
- Types explícitos (evitar `any`)

8.5 Testes

Unit Tests

```
# Go
cd core
go test ./...

# Frontend
cd frontend
npm test
```

Integration Tests

```
# Regtest
cd test/e2e
go test -tags=integration ./...
```

9. Referências

9.1 Documentação

Documento	Caminho	Descrição
Especificação Técnica	docs/XS_Wallet_Especificacao_Tecnica_v2.html	Fonte de verdade (humano)
Status vs Spec	docs/STATUS_ESPECIFICACAO_XS_WALLET.md	Gap analysis
Plano de Implementação	docs/PLANO_IMPLEMENTACAO_v0.2.md	Roadmap detalhado
Agent Guide	docs/AGENTS.md	Guia para agentes

9.2 Arquitetura

Componente	Caminho
Go Core	core/
Swap Engine	core/internal/swap/
Vault	core/internal/vault/
Boltz Client	core/internal/boltz/
Database	core/internal/db/
Frontend	frontend/
Protos	proto/

9.3 Recursos Externos

- Boltz API v2: api.docs.boltz.exchange
- BIP327 (MuSig2): github.com/bitcoin/bips
- LND gRPC: lightning.engineering/api-docs
- Elements RPC: elementsproject.org/en/doc